

CS/CE 224/227: Object Oriented Programming and Design Methodologies

Week 14/Unit 14: Templates

Faisal Alvi

(For any suggested modifications please email: faisal.alvi@sse.habib.edu.pk)



Unit Outline

1. Introduction
2. Templates
3. Summary

Templates: Motivation

- Consider the following function:

```
int abs(int n)                //absolute value of ints
{
    return (n<0) ? -n : n;    //if n is negative, return -n
}
```

- Instead of stating the same function for various data types such as:

```
long abs(long n)              //absolute value of longs
{
    return (n<0) ? -n : n;
}
```

```
float abs(float n)            //absolute value of floats
{
    return (n<0) ? -n : n;
}
```

Templates: Motivation

- We can make the function *generic* for any data type that operates on the operations “<” and “-”.
- Such a function can be written as:

```
template <class T>                //function template
T abs(T n)
{
    return (n < 0) ? -n : n;
}
```

- The statement `template<class T>` is known as a **function template** and the function is called a **template function**.
- The above function is *generic* in the sense that it is applicable for a generic type T.



Templates

- The key innovation in function templates is to represent the data type by a name that can stand for any type.
- In the preceding function template, this name is T.
- The template keyword signals the compiler that we're about to define a function template.
- The keyword class, within the angle brackets, might just as well be called type.
- The variable following the keyword class (T in this example) is called the template argument.



Templates

- Throughout the definition of the template we substitute the template argument, T.
- In the `abs()` template this name appears only twice, both in the first line (the function declarator), as the argument type and return type.
- In more complex functions it may appear numerous times throughout the function body as well.



Templates: How to call a template function?

```
int main()
{
    int int1 = 5;
    int int2 = -6;
    long lon1 = 70000L;
    long lon2 = -80000L;
    double dub1 = 9.95;
    double dub2 = -10.15;

    //calls instantiate functions
    cout << "\nabs(" << int1 << ")=" << abs(int1); //abs(int)
    cout << "\nabs(" << int2 << ")=" << abs(int2); //abs(int)
    cout << "\nabs(" << lon1 << ")=" << abs(lon1); //abs(long)
    cout << "\nabs(" << lon2 << ")=" << abs(lon2); //abs(long)
    cout << "\nabs(" << dub1 << ")=" << abs(dub1); //abs(double)
    cout << "\nabs(" << dub2 << ")=" << abs(dub2); //abs(double)
    cout << endl;
    return 0;
}
```



Templates: Multiple Arguments

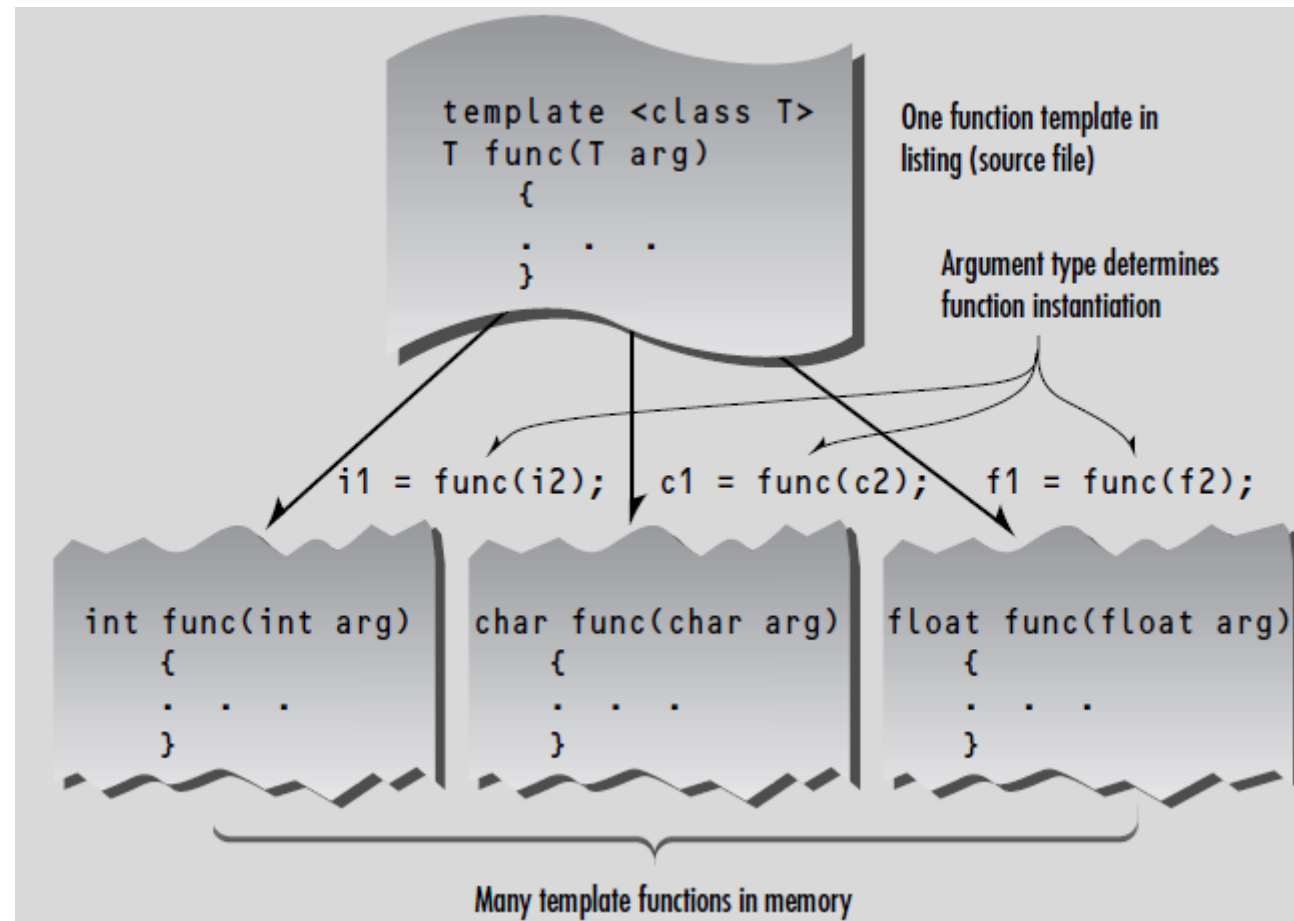
```
#include <iostream>
using namespace std;
// -----
//function returns index number of item, or -1 if not found
template <class atype>
int find(atype* array, atype value, int size)
{
    for(int j=0; j<size; j++)
        if(array[j]==value)
            return j;
    return -1;
}
// -----
```




Templates: Multiple Arguments

```
//-----  
char chrArr[] = {1, 3, 5, 9, 11, 13}; //array  
char ch = 5; //value to find  
int intArr[] = {1, 3, 5, 9, 11, 13};  
int in = 6;  
long lonArr[] = {1L, 3L, 5L, 9L, 11L, 13L};  
long lo = 11L;  
double dubArr[] = {1.0, 3.0, 5.0, 9.0, 11.0, 13.0};  
double db = 4.0;  
  
int main()  
{  
    cout << "\n 5 in chrArray: index=" << find(chrArr, ch, 6);  
    cout << "\n 6 in intArray: index=" << find(intArr, in, 6);  
    cout << "\n11 in lonArray: index=" << find(lonArr, lo, 6);  
    cout << "\n 4 in dubArray: index=" << find(dubArr, db, 6);  
    cout << endl;  
    return 0;  
}
```

Templates: Multiple Arguments





class Templates

```
// tempstak.cpp
// implements stack class as a template
#include <iostream.h>
using namespace std;
const int MAX = 100;           //size of array
////////////////////////////////////
template <class Type>
class Stack
{
private:
    Type st[MAX];              //stack: array of any type
    int top;                    //number of top of stack
public:
    Stack()                     //constructor
        { top = -1; }
    void push(Type var)         //put number on stack
        { st[++top] = var; }
    Type pop()                  //take number off stack
        { return st[top--]; }

};
////////////////////////////////////
```



class Templates

```
int main()
{
    Stack<float> s1;          //s1 is object of class Stack<float>

    s1.push(1111.1F);        //push 3 floats, pop 3 floats
    s1.push(2222.2F);
    s1.push(3333.3F);
    cout << "1: " << s1.pop() << endl;
    cout << "2: " << s1.pop() << endl;
    cout << "3: " << s1.pop() << endl;

    Stack<long> s2;          //s2 is object of class Stack<long>

    s2.push(123123123L);    //push 3 longs, pop 3 longs
    s2.push(234234234L);
    s2.push(345345345L);
    cout << "1: " << s2.pop() << endl;
    cout << "2: " << s2.pop() << endl;
    cout << "3: " << s2.pop() << endl;
    return 0;
}
```



Conclusion

- Templates provide generic versions of functions and classes.